

# Sri Lanka Institute of Information Technology



## Artificial Intelligence and Machine Learning | IT2011

**2025-Y2-S1-MLB-B7G1-02**

Topic Name: Student Stress Level Monitoring

Problem Domain: Healthcare

| Reg No     | Name                    |
|------------|-------------------------|
| IT24101574 | Dhanapala N. N.         |
| IT24103987 | Ilma M. S. F.           |
| IT24101571 | Balasooriya K. S. B.    |
| IT24101546 | De Silva G. H. T. D.    |
| IT24101618 | Rosayro M. C. J. De     |
| IT24101629 | Wijesundara W. M. B. H. |

## Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Introduction and Problem Statement.....</b>         | <b>2</b>  |
| <b>2. Dataset Description .....</b>                       | <b>2</b>  |
| 2.1 Repository Source .....                               | 2         |
| 2.2 Dataset Type and Size .....                           | 2         |
| <b>3. Preprocessing &amp; EDA .....</b>                   | <b>2</b>  |
| 3.1 Data Cleaning.....                                    | 2         |
| 3.2 Data Transformation.....                              | 4         |
| 3.3 Exploratory Data Analysis (EDA).....                  | 6         |
| 3.4 Final Preprocessed Data .....                         | 8         |
| <b>4. Model Design and Implementation .....</b>           | <b>8</b>  |
| 4.1 Objective .....                                       | 8         |
| 4.2 Models Used.....                                      | 8         |
| 4.3 Implementation steps.....                             | 9         |
| <b>5. Evaluation and Comparison.....</b>                  | <b>9</b>  |
| <b>6. Ethical Considerations and Bias Mitigation.....</b> | <b>10</b> |
| 6.1 Ethical Considerations.....                           | 10        |
| 6.2 Bias Mitigation Strategies .....                      | 10        |
| <b>7. Reflections and Lessons Learned.....</b>            | <b>11</b> |
| <b>8. References .....</b>                                | <b>12</b> |

## 1. Introduction and Problem Statement

Student mental health has become a growing concern across education in recent years. Social expectations, academic pressure, personal and environmental challenges contribute to elevated stress level among students, which can negatively impact their performance and long-term development

The goal of this project is to develop a Student Stress Level Monitoring System which predicts the stress categories like Low, Moderate and High using data on psychological, academic and social parameters. This model aims to support early detections while maintaining fairness and ethical data handling.

## 2. Dataset Description

### 2.1 Repository Source

- ✓ The dataset used in this project was sourced from [Kaggle's Student Stress Monitoring repository](#).
- ✓ Tabular data

### 2.2 Dataset Type and Size

- ✓ Type: Tabular
- ✓ Total Records: 1100 entries
- ✓ Total Attributes: 20 independent features representing different stress indicators, along with one target variable (stress\_level).

## 3. Preprocessing & EDA

### 3.1 Data Cleaning

- ✓ Removing null values

```
=== AFTER CLEANING ===  
Shape: (1100, 21)  
Missing values per column:  
anxiety_level      0  
self_esteem        0  
mental_health_history 0  
depression         0  
headache           0  
blood_pressure     0  
sleep_quality      0  
breathing_problem  0  
noise_level        0  
living_conditions  0  
safety             0  
basic_needs        0  
academic_performance 0  
study_load         0  
teacher_student_relationship 0  
future_career_concerns 0  
social_support     0  
peer_pressure      0  
extracurricular_activities 0  
bullying           0  
stress_level       0  
dtype: int64
```

Figure 1: Removing null values

✓ Removing Duplicate Values

```
dup_count_before = df_clean.duplicated().sum()
df_clean = df_clean.drop_duplicates()
```

```
Duplicates removed (pre-drop): 11
```

Figure 2: Removing Duplicate Values

✓ Removing outliers

```
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

usable_cols = IQR[IQR > 0].index.tolist()

if not usable_cols:
    print("All numeric columns have zero IQR (constant values). Skipping outl
    df_cleaned = df.copy()
    outlier_rows_removed = 0
else:
    lower_bounds = Q1[usable_cols] - 1.5 * IQR[usable_cols]
    upper_bounds = Q3[usable_cols] + 1.5 * IQR[usable_cols]

    within_lower = df[usable_cols].ge(lower_bounds, axis=1)
    within_upper = df[usable_cols].le(upper_bounds, axis=1)
    within_all = within_lower & within_upper

    condition = within_all.all(axis=1)

    df_cleaned = df[condition].copy()
    outlier_rows_removed = len(df) - len(df_cleaned)

print("Original shape:", df.shape)
print("After removing outliers:", df_cleaned.shape)
print(f"Rows removed as outliers: {outlier_rows_removed}")
```

Python

```
Original shape: (1100, 21)
After removing outliers: (793, 21)
Rows removed as outliers: 307
```

Figure 3: Removing outliers

## 3.2 Data Transformation

### ✓ Normalization/ Scaling

```
df_scaled = df.copy()
if numeric_cols:
    scaler = MinMaxScaler()
    df_scaled[numeric_cols] = scaler.fit_transform(df[numeric_cols])
else:
    print("No numeric columns found. Skipping scaling.")

print("\nHead of scaled data:")
display(df_scaled.head())
```

Head of scaled data:

|   | anxiety_level | self_esteem | mental_health_history | depression | headache | blood_pressure | sleep_quality | breathing_problem | noise_level | living_conditions |
|---|---------------|-------------|-----------------------|------------|----------|----------------|---------------|-------------------|-------------|-------------------|
| 0 | 0.666667      | 0.666667    | 0.0                   | 0.407407   | 0.4      | 0.0            | 0.4           | 0.8               | 0.333333    | 0.666667          |
| 1 | 0.714286      | 0.266667    | 1.0                   | 0.555556   | 1.0      | 1.0            | 0.2           | 0.8               | 0.666667    | 0.000000          |
| 2 | 0.571429      | 0.600000    | 1.0                   | 0.518519   | 0.4      | 0.0            | 0.4           | 0.4               | 0.333333    | 0.333333          |
| 3 | 0.761905      | 0.400000    | 1.0                   | 0.555556   | 0.8      | 1.0            | 0.2           | 0.6               | 1.000000    | 0.333333          |
| 4 | 0.761905      | 0.933333    | 0.0                   | 0.259259   | 0.4      | 1.0            | 1.0           | 0.2               | 0.666667    | 0.333333          |

5 rows x 11 columns

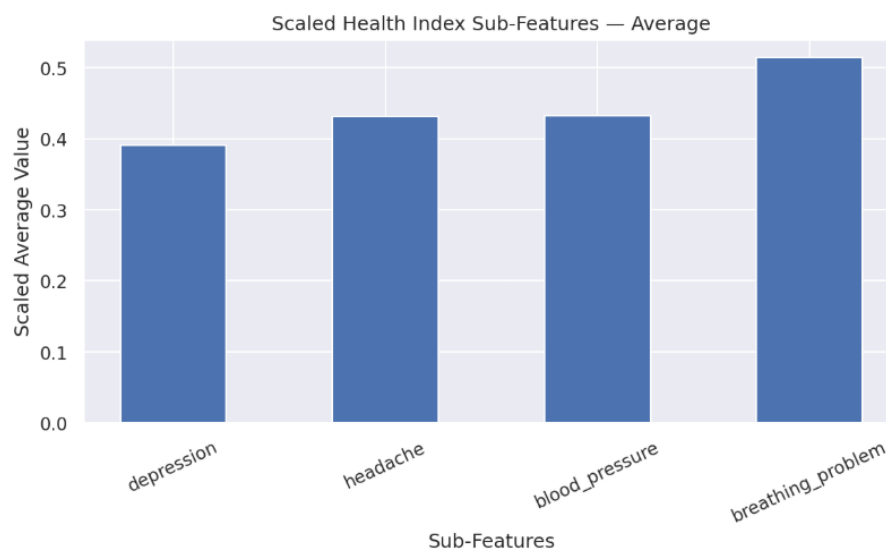
| basic_needs | academic_performance | study_load | teacher_student_relationship | future_career_concerns | social_support | peer_pressure | extracurricular_activities | bullying | stress_level |
|-------------|----------------------|------------|------------------------------|------------------------|----------------|---------------|----------------------------|----------|--------------|
| 0.4         | 0.6                  | 0.333333   | 0.6                          | 0.6                    | 0.666667       | 0.6           | 0.6                        | 0.4      | 1.0          |
| 0.4         | 0.2                  | 1.000000   | 0.2                          | 1.0                    | 0.333333       | 0.8           | 1.0                        | 1.0      | 2.0          |
| 0.4         | 0.4                  | 0.666667   | 0.6                          | 0.4                    | 0.666667       | 0.6           | 0.4                        | 0.4      | 1.0          |
| 0.4         | 0.4                  | 1.000000   | 0.2                          | 0.8                    | 0.333333       | 0.8           | 0.8                        | 1.0      | 2.0          |
| 0.6         | 0.8                  | 0.666667   | 0.2                          | 0.4                    | 0.333333       | 1.0           | 0.0                        | 1.0      | 1.0          |

Figure 4: Normalization/ Scaling

### ✓ Feature Creation

```
Created 'health_index' from: ['depression', 'headache', 'blood_pressure', 'breathing_problem']
Created 'academic_stress' from: ['academic_performance', 'study_load', 'future_career_concerns']
Created 'social_environment' from: ['social_support', 'peer_pressure', 'bullying']
```

Figure 5: Feature Creation



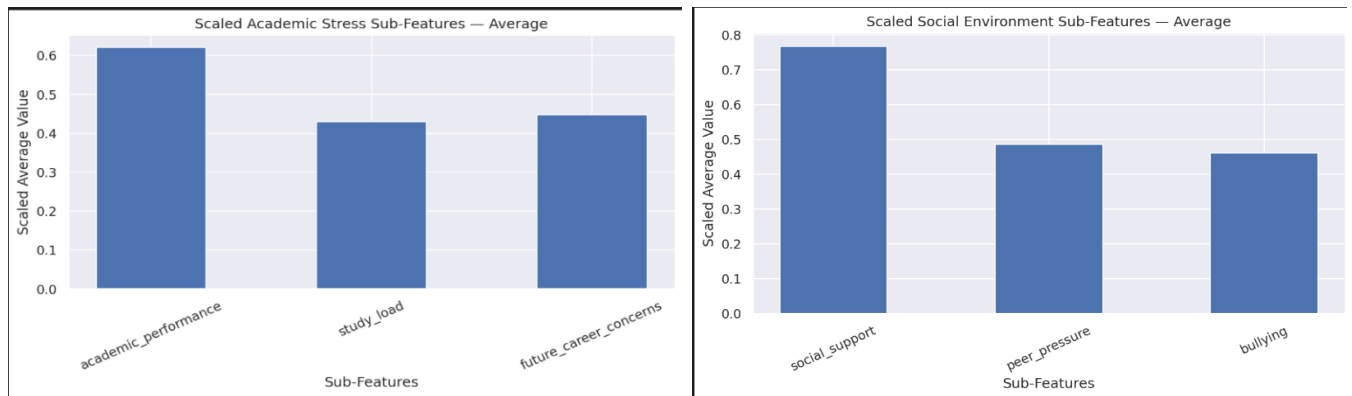


Figure 6: New features created

### ✓ Feature Selection

```

Feature Selection

Selecting top 10 features which is correlated with stress_level.

engineered_features = ["health_index", "academic_stress", "social_environment"]
# all columns except engineered ones
original_features = [col for col in df.columns if col not in engineered_features + ["stress_level"]]
# assuming 'stress_level' is your target variable

from sklearn.feature_selection import SelectKBest, f_classif

# Use the cleaned dataframe (df_cleaned) which has no missing values
X = df_cleaned[original_features] # only original features from cleaned data
y = df_cleaned["stress_level"]    # target variable from cleaned data

# select top 10 original features
selector = SelectKBest(score_func=f_classif, k=10)
X_new = selector.fit_transform(X, y)

# get selected feature names
selected_original = X.columns[selector.get_support()].tolist()
print("Top 10 selected original features:", selected_original)

Top 10 selected original features: ['self_esteem', 'headache', 'blood_pressure', 'sleep_quality', 'safety', 'basic_needs', 'academic_performance', 'teacher_student_relationship', 'f

```

Figure 7: Feature Selection

### ✓ Dimensionality Reduction

```

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Features only (exclude target)
X = df_final.drop("stress_level", axis=1)

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA with 5 components
pca = PCA(n_components=5)
X_pca = pca.fit_transform(X_scaled)

# Check explained variance ratio
print("Explained variance ratio of each component:", pca.explained_variance_ratio_)
print("Total explained variance by 5 components:", pca.explained_variance_ratio_.sum())

# Create a DataFrame with 5 PCs
df_pca = pd.DataFrame(X_pca, columns=["PC1", "PC2", "PC3", "PC4", "PC5"])
df_pca["stress_level"] = df_final["stress_level"]

# Optional: display head
print(df_pca.head())

plt.figure(figsize=(8,6))
scatter = plt.scatter(df_pca["PC1"], df_pca["PC5"], c=df_pca["stress_level"], cmap="viridis", alpha=0.7)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 5")
plt.title("PCA of Selected Features (PC1 vs PC5)")
plt.colorbar(scatter, label="Stress Level")
plt.show()

```

Figure 8: Dimensionality Reduction

### 3.3 Exploratory Data Analysis (EDA)

#### ✓ Correlation Heatmap of Features and Stress Level

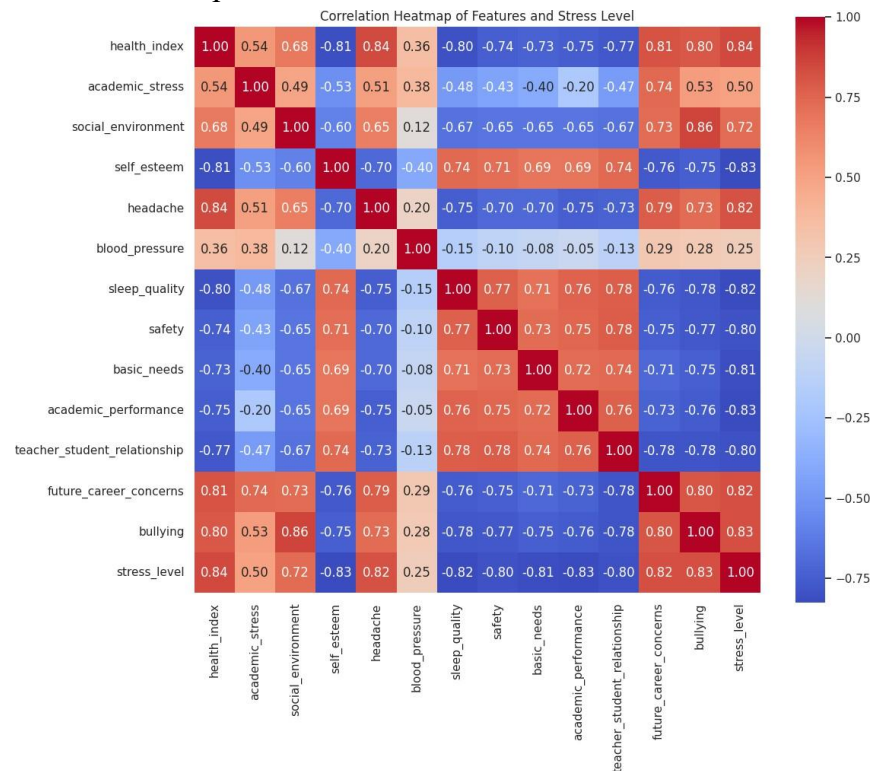


Figure 9: Correlation Heatmap of Features and Stress Level

#### ✓ Missing Value Heatmap

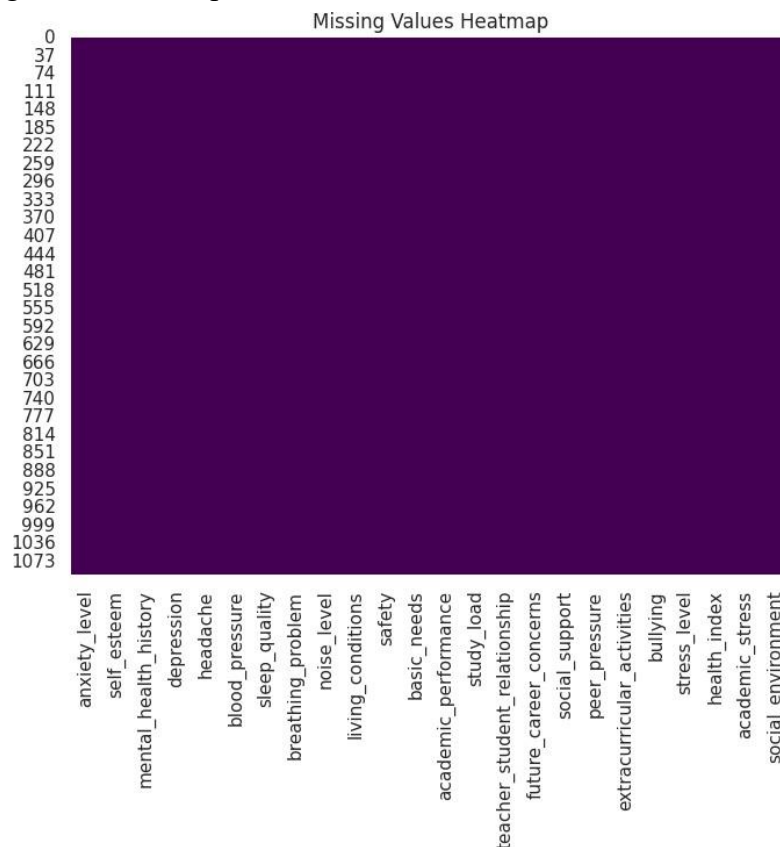


Figure 10: Missing Value Heatmap

✓ Histogram for numeric features

Histograms of Numerical Features

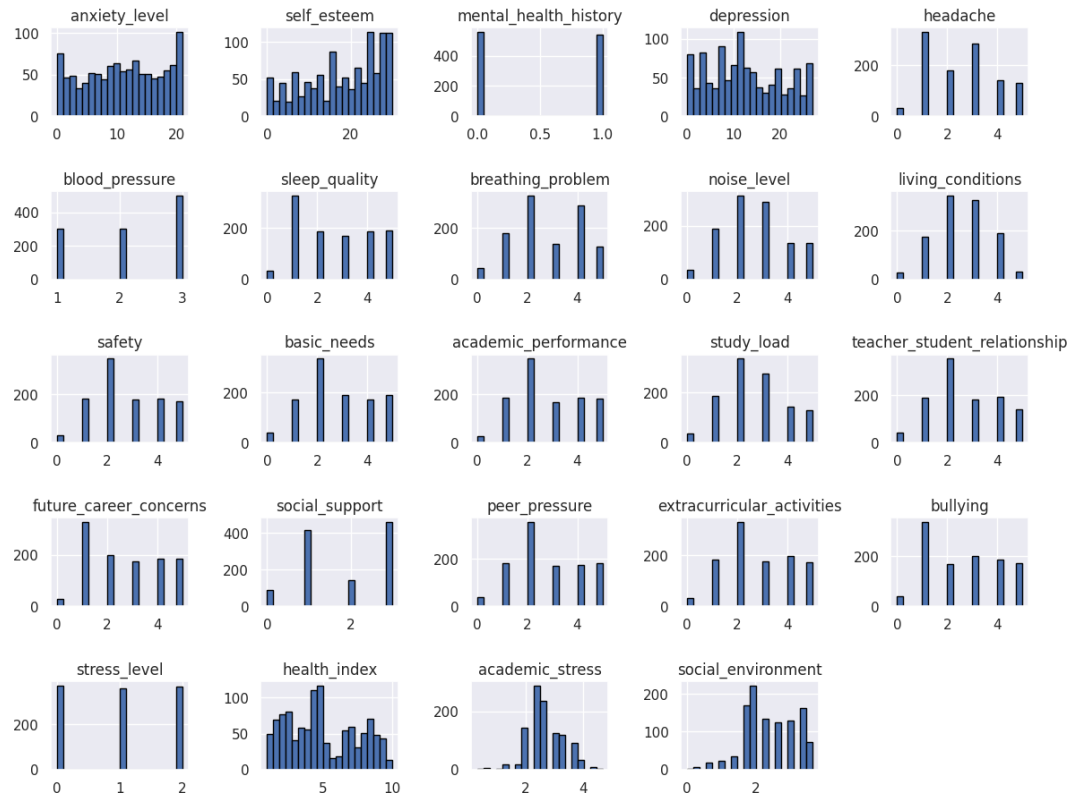


Figure 11: Histogram for numeric features

✓ Boxplot for numeric features

Boxplot of Numerical Features

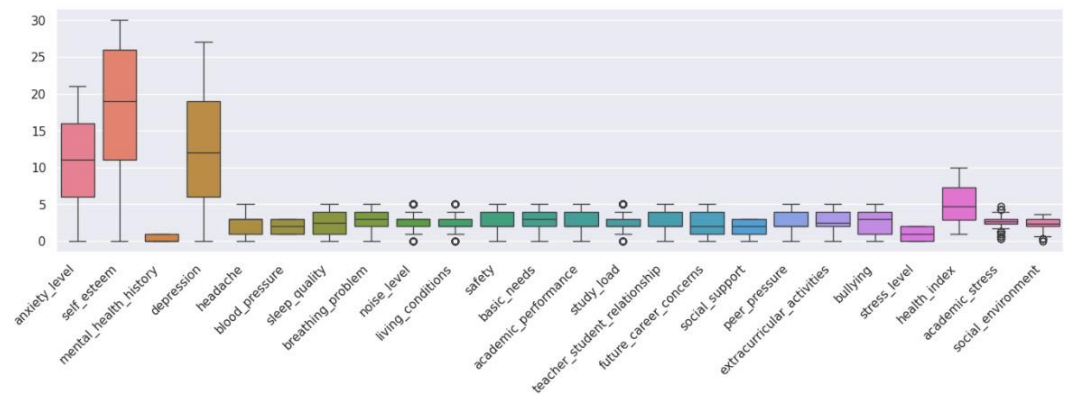


Figure 12: Boxplot for numeric features



### 3.4 Final Preprocessed Data

- ✓ After completing data cleaning and preprocessing, a final processed dataset was prepared for model training.
  - ✓ Null values and duplicate values removed
  - ✓ Outlier removal was done using Interquartile Range (IQR) method.
  - ✓ Numerical features were normalized.
  - ✓ Final features were selected based on correlation and relevance to stress level.
- 
- ✓ The target variable stress level was encoded into three classes:
    - 0 – Low Stress
    - 1 – Moderate Stress
    - 2 – High Stress

```
USE_PREPROCESSED = True
SKIP_SCALER_IF_PREPROCESSED = True
PREPROCESSED_PATH = "/content/drive/MyDrive/StudentStressLevelMonitoring/results/outputs/final_selected_features_dataset.csv"

df = pd.read_csv(PREPROCESSED_PATH)
assert 'stress_level' in df.columns, "Expected target 'stress_level'"
df = df.drop(columns=['id', 'student_id', 'fold', 'cv_pred', 'oof_pred'], errors='ignore')
df = df[df['stress_level'].notna()].copy()
y = df['stress_level'].astype(int)
X = df.drop(columns=['stress_level'])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print("Train shape:", X_train.shape, "| Test shape:", X_test.shape)
```

Figure 13: Using the preprocessed data to build the model

## 4. Model Design and Implementation

### 4.1 Objective

- ✓ To design models that predict and classify students' stress level into low, moderate and high categories based on the preprocessed data.

### 4.2 Models Used

- ✓ Logistic Regression
- ✓ RandomForest
- ✓ K-Nearest Neighbors (KNN)
- ✓ MLP
- ✓ XGBoost
- ✓ SVM

#### 4.3 Implementation steps

- ✓ Import necessary libraries
- ✓ Load the dataset
- ✓ Explore the dataset and target distribution
- ✓ Define features (X) and target (y)
- ✓ Split the dataset (80% training, 20% testing)
- ✓ Initialize the model
- ✓ Evaluate Baseline performance
- ✓ Hyperparameter Tuning using GridSearchCV
- ✓ Compare Baseline and Tuned models

#### 5. Evaluation and Comparison

- ✓ We selected multiple evaluation metrics rather than relying on a single measure.
  - i. Accuracy – Provides and overall measure of correct prediction across all classes
  - ii. Precision – Indicates the proportion of correctly predicted instances for each class which highlights the reliability of positive predictions
  - iii. Recall – Models ability to identify all actual instances of each class
  - iv. F1-Score – Combines precision and recall into a single metric

| Model               | Accuracy | Precision (macro) | Recall (macro) | F1-Score (macro) | AUC (OvR, macro) |
|---------------------|----------|-------------------|----------------|------------------|------------------|
| Logistic Regression | 0.97     | 0.97              | 0.96           | 0.97             | 0.99             |
| SVM (RBF)           | 0.97     | 0.97              | 0.96           | 0.97             | 0.99             |
| KNN                 | 0.95     | 0.94              | 0.94           | 0.94             | 0.98             |
| Random Forest       | 0.90     | 0.97              | 0.96           | 0.97             | 0.99             |
| XG Boost            | 0.96     | 0.96              | 0.95           | 0.96             | 0.99             |
| MLP                 | 0.94     | 0.94              | 0.94           | 0.94             | 0.98             |

## 6. Ethical Considerations and Bias Mitigation

Our **Student Stress Level Monitoring** project uses personal student data to predict stress level classes. We designed the workflow (cleaning → outlier removal → MinMax scaling → engineered features: *health index*, *academic stress*, *social environment* → SelectKBest → PCA → model) to be accurate **and** responsible.

### 6.1 Ethical Considerations

- I. Data Privacy: Protect sensitive information and anonymize data to prevent misuse.
- II. Transparency: Clearly communicate what the model does, its limitations, and uncertainties.
- III. Fairness: Avoid models that systematically disadvantage certain groups (by age, gender, or ethnicity).
- IV. Accountability: Ensure decisions made by models can be explained and audited.
- V. Avoid Harm: Models should not be used for high-stakes decisions without proper validation.

### 6.2 Bias Mitigation Strategies

- I. Feature Selection: Exclude or carefully handle features that could introduce bias.
- II. Data Balancing: Address class imbalances to prevent skewed predictions.
- III. Preprocessing: Standardize or normalize features to avoid unintentional weighting.
- IV. Evaluation Across Groups: Check metrics like precision, recall, or F1-score for fairness across subgroups.
- V. Model Transparency: Use interpretable models or explainability tools to ensure decisions are understandable.

## 7. Reflections and Lessons Learned

- ✓ This project provided insights into applying real world problem to develop a predictive model.
  - Clean data = better results. Removing missing values, duplicates, and outliers made every model more stable.
  - Simple, meaningful features help. Our engineered features—*health\_index*, *academic stress*, *social environment*—captured useful patterns without being complicated
  - Use the right metrics. We looked beyond accuracy to macro Precision, Recall, and F1, so each stress class is treated fairly. We paid special attention to recall for High stress.
  - **Save and compare.** Storing results.json, CV tables, and confusion matrices made it easy to compare models and write the report.

## 8. References

- [1] "Kaggle," [Online]. Available: [https://www.kaggle.com/code/vipulgandhi/pca-beginner-s-guide-to-dimensionality-reduction?utm\\_source=chatgpt.com](https://www.kaggle.com/code/vipulgandhi/pca-beginner-s-guide-to-dimensionality-reduction?utm_source=chatgpt.com). [Accessed 10 2025].
- [2] "Kaggle," [Online]. Available: <https://www.kaggle.com/models?owner-type=organization>. [Accessed 10 2025].
- [3] "scikitlearn," [Online]. Available: [https://scikit-learn.org/stable/modules/cross\\_validation.html#cross-validation-and-model-selection](https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-and-model-selection). [Accessed 2025].
- [4] "Medium," [Online]. Available: <https://medium.com/@murpanironit/a-comprehensive-guide-to-multiclass-classification-in-machine-learning-c4f893e8161d>. [Accessed 10 2025].
- [5] "GeekforGeeks," [Online]. Available: <https://www.geeksforgeeks.org/artificial-intelligence/aiml-introduction/>. [Accessed 10 2025].